Chat Transcript: Pac-Man game.py fixes

Date: Wednesday, October 28, 2026. Conversation covers four tasks on *game.py*.

You

Task 1: Fix the power pellet bug. Eating a power pellet is supposed to send every ghost into frightened mode for a few seconds, reversing their direction and making them vulnerable. In the current build, eating a power pellet does nothing beyond the usual 10 points; frightened mode never triggers. The maze layout marks power pellets with one specific character; check what character `eat()` compares a cell against, versus what the maze grid actually uses for power pellets.

Claude

The bug was a case mismatch. The maze grid marks power pellets with a lowercase `"o"`, but `eat()` checked for an uppercase `"O"`, so the check never matched and the frightened-mode code never ran.

The fix is one line in `eat()` in `game.py`:

```
if MAZE[cell[0]][cell[1]] == "o":
```

With that change, eating a power pellet adds the 40-point bonus, starts the frightened timer, and reverses every ghost that hasn't already been eaten. I checked that the file compiles but haven't played it.

You

Task 2: Implement `ghost_color(name, mode)`**.** Called once per ghost per frame in `draw`, as `color = ghost_color(ghost.name, mode) or color`, after the default color has already been computed. `name` is one of blinky, pinky, inky, clyde; `mode` is normal, frightened, or eaten. Return an (r, g, b) override, or None to keep the default. Idea: give each ghost a distinct frightened-mode tint instead of the shared blue/white flash.

Claude

I implemented `ghost_color` in `game.py`. In frightened mode each ghost now gets its own tint, taken from a new `FRIGHT_TINTS` table next to the other constants. In normal and eaten modes it returns `None`, so those keep the default colors. I checked that it compiles and returns the expected values for all four ghosts, but I haven't run the game.

The tints are purple for Blinky, blue for Pinky, teal for Inky and magenta for Clyde.

Because `ghost_color` always returns a tint while a ghost is frightened, it overrides the default white flash that warns the player when frightened mode is about to end. The function isn't told how much frightened time is left, so it can't flash on its own.

To keep that warning, I'd either flash the tint inside `ghost_color` using `pygame.time.get_ticks()`, or change the `draw` line to skip the override when the default is the white flash. Say which you prefer and I'll make the change.

You

Task 3: Implement `on_pellet_eaten(score, pellets_left)`**.** Called from `eat()` immediately after every pellet (regular or power) is removed from the board and the score updated. It receives the score after this pellet's points were added, and the number of pellets still remaining. Its return value is ignored. Idea: spawn a bonus fruit once `pellets_left` crosses a specific number, or flash the HUD on the final pellet.

Claude

I implemented `on_pellet_eaten` in `game.py` as a HUD flash: the score line blinks yellow for 1.5 seconds when 10 pellets are left and again on the final pellet. The file compiles, but I haven't run the game.

The hook only receives `score` and `pellets_left`, and its return value is ignored. It can't reach the `Game` object, so it works through a module-level timestamp, `hud_flash_until`, which `draw` reads to pick the HUD color. I made three small supporting edits:

• **New constants:** `HUD_FLASH_MS`, `HUD_FLASH_AT` and `hud_flash_until` sit next to the other constants. Change the `HUD_FLASH_AT` set to flash at different pellet counts.

• `Game.reset`**:** it clears the timer, so pressing R doesn't carry a flash into the new game.

• `draw`**:** it now picks the HUD color, and the flash toggles every 150 ms.

The final-pellet flash shows on the "YOU WIN!" screen, since `draw` keeps running after the win state is set.

I chose the flash over a bonus fruit because a fruit needs a position, a timer and collision handling inside `Game`. That would mean changing more than this hook. I can add it if you want.

You

Task 4: Implement `bonus_life_threshold()`**.** Called every frame in `update()`. It takes no arguments and should return an integer score value, or None to disable bonus lives entirely. Whenever the player's score crosses a multiple of that value for the first time, one life is awarded automatically; the surrounding bookkeeping (`self.bonus_awarded`) is already implemented, so you only need to choose or compute the threshold. Idea: return a fixed value like 10000.

Claude

`bonus_life_threshold()` in `game.py` now returns `1000`, set through a new `BONUS_LIFE_SCORE` constant. The file compiles, but I haven't run the game.

I didn't use the suggested `10000` because it could never be reached. This maze has 126 regular pellets worth 10 points each and 4 power pellets worth 50 each (10 plus the 40 bonus), so clearing the board gives about 1,460 points. Eating ghosts adds 200 each, so a good run might add a few hundred more. A threshold of 1000 should give most players one extra life late in a run, and a strong run might earn a second. If you want it to be harder to earn, around 1200 would make it rarer.

The existing bookkeeping in `update()` handles the rest. It awards one life per multiple of the threshold, and pressing R resets `bonus_awarded`. The bonus doesn't fire on a win, because `update()` stops running once the state leaves "play".

You

Give me pdf of the chat until now.